SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : SHIVAPURAM AMRUTHA

Reg. No : 11249A347

Class : II B.E. (CSE)

Course Code: BCSF244T40

Course Name: Design and Analysis of Algorithm

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms. SHIVAPURAM AMRUTHA ,**

with **Reg. No 11249A347** of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station: Enathur

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on _____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	Program Name	Date	Page no	Signature

SL.NO	Program Name	Date	Page no	Signature

Exp.No: 1

Date: 27.01.2026

Implement of Tower Of Hanoi

AIM:

To write and execute a c program to solve the tower of hanoi problem using recursion.

ALGORITHM:

Step 1: Start the program
Step 2: Read the number of disks n
Step 3: Define a recursive function tower (n, source, auxiliary, destination)
Step 4: If n == 1:
*Move the disk from source to destination.
*Return
Step 5: Otherwise
*Move n-1 disks from source to auxiliary using destination as helper
*Move the nth disk from source to destination.
*Move the n-1 disks from auxiliary to destination using source as helper.
Step 6 : Repeat until all disks are moved
Step 7 : Stop the program

PROGRAM :

```
#include <stdio.h>

void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod) {
   if (n == 1) {
      printf("Move disk 1 from rod %c to rod %c\n", from_rod, to_rod);
      return;
   }
     towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);

   printf("Move disk %d from rod %c to rod %c\n", n, from_rod, to_rod);

      towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
}

int main() {
   int n = 3;  // Number of disks
   printf("Steps to solve Tower of Hanoi with %d disks:\n", n);
   // A: Source, C: Destination, B: Auxiliary
   towerOfHanoi(n, 'A', 'C', 'B');
   return 0;
}
```

OUTPUT:

Steps to solve Tower of Hanoi with 3 disks:
Move disk 1 from rod A to rod C
Move disk 2 from rod A to rod B
Move disk 1 from rod C to rod B
Move disk 3 from rod A to rod C
Move disk 1 from rod B to rod A
Move disk 2 from rod B to rod C
Move disk 1 from rod A to rod C

RESULT:

Thus the implementation of Tower of Hanoi was successfully implemented and verified.

Ex.No: 2
Date : 03.02.2026

Implement nth Fibonacci term using recursion & iteration

AIM:
To calculate a specific number at the nth position in the Fibonacci sequence using recursion & iteration method.
ALGORITHM:

i) Using recursion
Step 1: Start the program
Step 2: Read the value of n
Step 3: Define recursive function fib(n).
Step 4: If n = = 0, return 0
Step 5: If n = = 1, return 1
Step 6: Otherwise, return fib(n-1) + fib(n-2)
Step 7: Call the function with n
Step 8:Display the nth fibonacci term
Step 9: Stop the program

ii) Using iteration
Step 1: Start the program
Step 2: Read the value of n
Step 3: Initialize variables:
*a = 0, b = 1
Step 4: If n = = 0, return a
Step 5: If n = = 1, return b
Step 6: Use a loop from 2 to n:
* c = a+b *a=b *b=c
Step 7: After the loop, b conatins the nth Fibonacci term.
Step 8:Display the result
Step 9: Stop the program

PROGRAM :

i) Using recursion

```
#include <stdio.h>

// Function prototype for the recursive function
int fibonacci(int n);

int main() {
   int n, result;

   printf("Enter the nth number in the Fibonacci series: ");
   scanf("%d", &n);
```

```
    if (n < 0) {
        printf("Fibonacci of a negative number is not possible.\\n");
    } else {
        result = fibonacci(n);
        printf("The %dth number in the Fibonacci series is %d\\n", n, result);
    }

    return 0;
}

// Recursive function to calculate the nth Fibonacci number
int fibonacci(int n) {
    // Base cases: F(0) = 0, F(1) = 1
    if (n <= 1) {
        return n;
    }
    // Recursive step: F(n) = F(n-1) + F(n-2)
    else {
        return (fibonacci(n - 1) + fibonacci(n - 2));
    }
}
```

ii)Using iteration

```
#include <stdio.h>

int main() {
    int n, i;
    long long first = 0, second = 1, next;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    if (n <= 1) {
        printf("Fibonacci term %d is: %d\n", n, n);
    } else {
        for (i = 2; i <= n; i++) {
            next = first + second;
            first = second;
            second = next;
        }
        printf("Fibonacci term %d is: %lld\n", n, second);
    }

    return 0;
}
```

OUTPUT:

i) Using recursion

Enter the term numer (n) to find the nth Fibonacci term : 10

The 10th Fibonacci term is : 55

ii) Using iteration

Enter the value of n: 10
Fibonacci term 10 is: 55

RESULT:

The implement nth Fibonacci term using recursion & iteration was successfully implemented and verified.

Ex.No: 3
Date: 10.02.2026

Implement Bin Packing

AIM:
To pack a set of items of various sizes into the minimum number of bins possible, where each bin has a fixed capacity, without exceeding that capacity

ALGORITHM:
Step 1: Start with the first item
Step 2: Check the bins in order
Step 3: Place the item in the first bin that has enough remaining space
Step 4: If no existing bin fit the item, start a new bin

PROGRAM :

```
#include<stdio.h>

void binPacking(int *a, int size, int n) {
   int binCount = 1, i;
   int s = size;
   for (i = 0; i < n; i++) {
      if (s - *(a + i) >= 0) {
         s -= *(a + i);
         continue;
      } else {
         binCount++;
         s = size;
         i--;
      }
   }
   printf("Number of bins required: %d\n", binCount);
}

int main(int argc, char **argv) {
   printf("Enter the number of items in Set: ");
   int n;
   scanf("%d", &n);
   int a[n], i;
   printf("Enter %d items: ", n);
   for (i = 0; i < n; i++) {
      scanf("%d", &a[i]);
   }
   printf("Enter the bin size: ");
   int size;
   scanf("%d", &size);
   binPacking(a, size, n);
   return 0;
}
```

OUTPUT:

Enter the number of items in Set: 7

Enter 7 items: 2 5 4 7 1 3 8
Enter the bin size: 10
Number of bins required: 4

RESULT:

The implementation of Bin Packing was successfully implemented and verified.

Ex.No: 4
Date: 02.03.2026

Implement Fractional Knapsack using greedy method

AIM:
To maximize profit by selecting items (including fractions) within a given capacity using the **Fractional Knapsack Algorithm**.

ALGORITHM:
Step 1: Compute profit/weight ratio for each item.
Step 2: Sort items in decreasing order of ratio.
Step 3: Pick items fully while capacity allows.
Step 4: If an item cannot be taken fully, take the fractional part.
Step 5: Stop when the knapsack is full.

PROGRAM:
```
#include <stdio.h>

int main() {
   int n = 3, i, W = 50;
   float profit[] = {60, 100, 120};
   float weight[] = {10, 20, 30};
   float ratio[3], temp, maxProfit = 0;

   // Calculate ratio
   for(i = 0; i < n; i++)
      ratio[i] = profit[i] / weight[i];

   // Sort by ratio (descending)
   for(i = 0; i < n - 1; i++) {
      for(int j = i + 1; j < n; j++) {
         if(ratio[i] < ratio[j]) {
            temp = ratio[i]; ratio[i] = ratio[j]; ratio[j] = temp;

            temp = profit[i]; profit[i] = profit[j]; profit[j] = temp;

            temp = weight[i]; weight[i] = weight[j]; weight[j] = temp;
         }
      }
   }

   // Knapsack logic
   for(i = 0; i < n; i++) {
      if(W >= weight[i]) {
         maxProfit += profit[i];
         W -= weight[i];
      } else {
         maxProfit += ratio[i] * W;
         break;
      }
   }
```

```
    printf("Maximum Profit = %.2f", maxProfit);
    return 0;
}
```

OUTPUT :

Maximum Profit = 240.00

RESULT:

The implementation of Fractional Knapsack using greedy method was successfully implemented and verified.

Ex.No: 5
Date: 17.03.2026

Implement Travelling Salesman Problem

AIM:
To find the minimum cost tour that visits all cities exactly once and returns to the starting city using the **Travelling Salesman Problem**.

ALGORITHM:
Step 1: Start from a chosen city..
Step 2: Visit all possible permutations of remaining cities.
Step 3: Calculate total cost for each tour..
Step 4: Select the tour with minimum cost.
Step 5: Return to the starting city.

PROGRAM:
```
#include <stdio.h>
#include <limits.h>

#define V 4

int min(int a, int b) {
   return (a < b) ? a : b;
}

// TSP using recursion
int tsp(int graph[V][V], int visited[], int curr, int count, int cost, int ans) {
   if (count == V && graph[curr][0]) {
      return min(ans, cost + graph[curr][0]);
   }

   for (int i = 0; i < V; i++) {
      if (!visited[i] && graph[curr][i]) {
         visited[i] = 1;
         ans = tsp(graph, visited, i, count + 1,
                   cost + graph[curr][i], ans);
         visited[i] = 0;
      }
   }
   return ans;
}

int main() {
   int graph[V][V] = {
      {0, 10, 15, 20},
      {10, 0, 35, 25},
      {15, 35, 0, 30},
      {20, 25, 30, 0}
   };

   int visited[V] = {0};
```

```
    visited[0] = 1;

    int ans = tsp(graph, visited, 0, 1, 0, INT_MAX);

    printf("Minimum Cost = %d", ans);

    return 0;
}
```

OUTPUT:
Minimum Cost = 80

RESULT:
The implementation of Travelling Salesman Problem was successfully implemented and verified.

Ex.No: 6
Date: 31.03.2026

Implement Minimum Spanning Tree

AIM:
To connect all vertices of a weighted, connected graph with **minimum total edge weight** without forming cycles, using a **Minimum Spanning Tree (MST)**

ALGORITHM:
Step 1: Start with any vertex and mark it as part of MST.
Step 2: Find the smallest edge connecting a vertex in MST to a vertex outside MST.
Step 3: Add that vertex and edge to MST
Step 4: Repeat step 2 until all vertices are included

PROGRAM:
```
#include <stdio.h>
#include <limits.h>
#define MAX_VERTICES 100

// Function to find the vertex with the minimum key value
int minKey(int key[], int mstSet[], int vertices) {
    int min = INT_MAX, minIndex;
    for (int v = 0; v < vertices; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            minIndex = v;
        }
    }
    return minIndex;
}

// Function to print the constructed MST stored in parent[]
void printMST(int parent[], int graph[MAX_VERTICES][MAX_VERTICES], int vertices) {
    printf("Edge \tWeight\n");
    for (int i = 1; i < vertices; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

// Function to implement Prim's algorithm for a given graph
void primMST(int graph[MAX_VERTICES][MAX_VERTICES], int vertices) {
    int parent[MAX_VERTICES]; // Array to store the constructed MST
    int key[MAX_VERTICES]; // Key values used to pick the minimum weight edge
    int mstSet[MAX_VERTICES]; // To represent set of vertices included in MST

    // Initialize all keys as INFINITE and mstSet[] as false
    for (int i = 0; i < vertices; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
```

```
    }

    // Always include the first vertex in the MST
    key[0] = 0; // Make key 0 so that this vertex is picked as the first vertex
    parent[0] = -1; // First node is always the root of the MST

    // The MST will have vertices-1 edges
    for (int count = 0; count < vertices - 1; count++) {
        // Pick the minimum key vertex from the set of vertices not yet included in the MST
        int u = minKey(key, mstSet, vertices);

        // Add the picked vertex to the MST Set
        mstSet[u] = 1;

        // Update key value and parent index of the adjacent vertices
        for (int v = 0; v < vertices; v++) {
            // graph[u][v] is non-zero only for adjacent vertices of u
            // mstSet[v] is false for vertices not yet included in MST
            // Update the key only if graph[u][v] is smaller than key[v]
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // printMST(parent, graph, vertices);
}

int main() {
    int vertices;

    // Input the number of vertices
    printf("Input the number of vertices: ");
    scanf("%d", &vertices);

    if (vertices <= 0 || vertices > MAX_VERTICES) {
        printf("Invalid number of vertices. Exiting...\n");
        return 1;
    }

    int graph[MAX_VERTICES][MAX_VERTICES];

    // Input the adjacency matrix representing the graph
    printf("Input the adjacency matrix for the graph:\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Perform Prim's algorithm to find the MST
    primMST(graph, vertices);

    return 0;
```

}

OUTPUT:

Edge	Weight
0-1	2
1-2	3
0-3	6
1-4	5

RESULT:

The implementation of Minimum Spanning Tree was successfully implemented and verified.

Ex.No: 7
Date: 31.03.2026

Implement Shortest path algorithm

AIM:
To find the shortest (minimum cost) path between a source node and all other nodes in a weighted graph using Dijkstra's Algorithm

ALGORITHM:
Step 1: Initialize distances of all vertices as ∞, set source = 0.
Step 2: Mark all vertices as unvisited.
Step 3: Pick the vertex with the smallest distance.
Step 4: Update distances of its adjacent vertices.
Step 5: Mark the vertex as visited.
Step 6: Repeat steps 3–5 until all vertices are visited

PROGRAM:
```
#include <stdio.h>
#include <limits.h>

#define V 5  // Number of vertices

// Find vertex with minimum distance
int minDistance(int dist[], int visited[]) {
   int min = INT_MAX, min_index = -1;

   for (int i = 0; i < V; i++) {
      if (!visited[i] && dist[i] < min) {
         min = dist[i];
         min_index = i;
      }
   }
   return min_index;
}

// Print shortest distances
void printSolution(int dist[]) {
   printf("Vertex\tDistance from Source\n");
   for (int i = 0; i < V; i++) {
      printf("%d\t%d\n", i, dist[i]);
   }
}

// Dijkstra Algorithm
void dijkstra(int graph[V][V], int src) {
   int dist[V], visited[V];

   // Initialize
   for (int i = 0; i < V; i++) {
      dist[i] = INT_MAX;
      visited[i] = 0;
   }
```

```
    dist[src] = 0;

    // Find shortest path
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printSolution(dist);
}

// Main function
int main() {
    int graph[V][V] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
        {100, 0, 10, 60, 0}
    };

    int source = 0;
    dijkstra(graph, source);

    return 0;
}
```

OUTPUT:

```
Vertex   Distance from Source
0        0
1        10
2        50
3        30
4        60
```

RESULT:

The implementation of Shortest Path Algorithm was successfully implemented and verified.

Ex.No: 8
Date: 31.03.2026

Implement Network Flow algorithm

AIM:
To implement the Network Flow algorithm to find the maximum flow from a source to a sink in a graph

ALGORITHM:
Step 1: Initialize flow = 0.
Step 2: Find an augmenting path from source to sink.
Step 3: Find minimum residual capacity in that path.
Step 4: Add this flow to total flow.
Update residual graph.
Step 5: Repeat until no augmenting path exists.
Step 6: Output maximum flow.

PROGRAM:
```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 6  // Number of vertices

// BFS to find augmenting path
bool bfs(int rGraph[V][V], int s, int t, int parent[]) {
    int visited[V] = {0};
    int queue[V], front = 0, rear = 0;

    queue[rear++] = s;
    visited[s] = 1;
    parent[s] = -1;

    while (front < rear) {
        int u = queue[front++];

        for (int v = 0; v < V; v++) {
            if (!visited[v] && rGraph[u][v] > 0) {
                queue[rear++] = v;
                parent[v] = u;
                visited[v] = 1;
            }
        }
    }
    return visited[t];
}

// Ford-Fulkerson Algorithm
int fordFulkerson(int graph[V][V], int s, int t) {
    int u, v;
    int rGraph[V][V];
```

```
    for (u = 0; u < V; u++)
        for (v = 0; v < V; v++)
            rGraph[u][v] = graph[u][v];
    int parent[V];
    int max_flow = 0;
    while (bfs(rGraph, s, t, parent)) {
        int path_flow = INT_MAX
        for (v = t; v != s; v = parent[v]) {
            u = parent[v];
            if (rGraph[u][v] < path_flow)
                path_flow = rGraph[u][v];
        }
        for (v = t; v != s; v = parent[v]) {
            u = parent[v];
            rGraph[u][v] -= path_flow;
            rGraph[v][u] += path_flow;
        }
        max_flow += path_flow;
    }
    return max_flow;
}
int main() {
    int graph[V][V] = {
        {0, 16, 13, 0, 0, 0},
        {0, 0, 10, 12, 0, 0},
        {0, 4, 0, 0, 14, 0},
        {0, 0, 9, 0, 0, 20},
        {0, 0, 0, 7, 0, 4},
        {0, 0, 0, 0, 0, 0}
    };
    int source = 0, sink = 5;
    printf("Maximum Flow = %d\n", fordFulkerson(graph, source, sink));
    return 0;
}
```

OUTPUT:

Maximum Flow = 23

RESULT:

The implementation of Network Flow algorithm was successfully implemented and verified.

Ex.No: 9
Date: 31.03.2026

Implement Approximation algorithms

AIM:
To implement an approximation algorithm to find a near-optimal solution to the Vertex Cover Problem in a graph, which is NP-hard

ALGORITHM:
Step 1: Start with an empty vertex cover set
Step 2: While there are uncovered edges in the graph:
   a) Pick any uncovered edge (u,v)
   b) Add both u and v to c
   c) Remove all edges incident to u or v
Step 3:Return c as the approximate vertex cover.
Step 4: Stop

PROGRAM:
```
#include <stdio.h>
#include <stdbool.h>

#define V 6  // Number of vertices

int main() {
   int graph[V][V] = {
      {0,1,1,0,0,0},
      {1,0,0,1,0,0},
      {1,0,0,1,1,0},
      {0,1,1,0,1,1},
      {0,0,1,1,0,1},
      {0,0,0,1,1,0}
   };

   bool visited[V] = {false};
   int u, v;

   printf("Approximate Vertex Cover: ");

   for (u = 0; u < V; u++) {
      for (v = 0; v < V; v++) {
         if (graph[u][v] && !visited[u] && !visited[v]) {
            visited[u] = visited[v] = true;
            printf("%d %d ", u, v);
         }
      }
   }

   printf("\n");
   return 0;
}
```

OUTPUT:
Approximate Vertex Cover: 0 1 2 3 4 5

RESULT:
The implementation of Approximation algorithms was successfully implemented and verified.

Ex.No: 10
Date: 31.03.2026

Implement Randomized algorithm

AIM:
To implement a randomized algorithm that uses random choices to solve a problem efficiently, possibly with probabilistic guarantees, such as finding a random subset, random graph selection, or randomized quicksort.

ALGORITHM:
Step 1: Identify the problem to solve (e.g., selection, optimization, graph problem).
Step 2: Make one or more random choices in the algorithm (e.g., pick a random element or edge).
Step 3: Perform deterministic operations based on the random choices.
Step 4: Repeat or iterate if necessary to improve probability of success.
Step 5: Output the result; it is correct or approximate with high probability

PROGRAM:
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define V 4
#define E 5

typedef struct {
   int u, v;
} Edge;

// Find set of a vertex (for union-find)
int find(int parent[], int i) {
   if (parent[i] == -1)
      return i;
   return find(parent, parent[i]);
}

// Union of two sets
void unionSets(int parent[], int x, int y) {
   int xset = find(parent, x);
   int yset = find(parent, y);
   parent[xset] = yset;
}

// Randomized Min Cut using Karger's Algorithm
int randomizedMinCut(Edge edges[], int V, int E) {
   int parent[V];
   for (int i = 0; i < V; i++)
      parent[i] = -1;
```

```
    int vertices = V;

    srand(time(0));

    while (vertices > 2) {
        int randEdge = rand() % E;
        int uSet = find(parent, edges[randEdge].u);
        int vSet = find(parent, edges[randEdge].v);

        if (uSet == vSet)
            continue;

        unionSets(parent, uSet, vSet);
        vertices--;
    }

    int cutEdges = 0;
    for (int i = 0; i < E; i++) {
        int uSet = find(parent, edges[i].u);
        int vSet = find(parent, edges[i].v);
        if (uSet != vSet)
            cutEdges++;
    }

    return cutEdges;
}

int main() {
    Edge edges[E] = {{0,1},{0,2},{1,2},{1,3},{2,3}};

    int minCut = randomizedMinCut(edges, V, E);

    printf("Approximate Minimum Cut = %d\n", minCut);

    return 0;
}
```

OUTPUT:

Approximate Minimum Cut = 2

RESULT:

The implementation of Randomized algorithm was successfully implemented and verified.